IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project –High Level Design

on

Data driven risk profiling for finance

Student Name: Namith J Shetty

Industry Mentor:
Mr.Sumith Kumar Shukla

Guided by:Ms Prathiksha

Table of Contents

1. Introduction

2. System Design

2.1 Application Design

2.2 Process Flow

2.3 Information Flow

2.4 Components Design

2.5 Key Design Considerations

2.6 API Catalogue

3. Data Design

3.1 Data Model

3.2 Data Access Mechanism

3.3 Data Retention Policies

3.4 Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non Functional Requirements

7.1 Security Aspects

7.2 Performance Aspects

8. References

1. Introduction

FinRisk Pro is a web based financial risk assessment platform that delivers real time, explainable risk profiling. It is built using Streamlit for the UI, Pandas and NumPy for data processing, and Plotly for interactive visualisations. The system targets banks, insurers, and fintech analysts who need transparent, auditable risk decisions.

2. System Design

2.1 Application Design

The application follows a modular, object oriented Python structure, separating core logic (src/risk_engine.py), visualisation (src/visualization.py), data loading (src/data_loader.py), preprocessing (src/preprocessing.py), and constants (src/constants.py). The main Streamlit script (app.py) orchestrates the UI and ties all modules together.

2.2 Process Flow

1. **User uploads a CSV or activates sample data.**
2. **System validates and loads data into a Pandas DataFrame.**
3. **Preprocessing cleans and normalises the 8 risk features.**
4. **Risk Engine computes weighted base scores and applies business rules.**
5. **Results are stored in the DataFrame; risk categories and decisions are assigned.**
6. **UI renders KPI cards, individual customer drill down, portfolio level charts, and XAI panels.**
7. **User can adjust weights or filters; the engine recomputes instantly.**
8. **Export functionality generates downloadable reports.**

2.3 Information Flow

- **User → App: File upload, slider adjustments, filter selections.**
- **App → Data Loader: Raw CSV bytes → DataFrame.**
- **Preprocessing → Risk Engine: Normalised feature matrix.**

- **Risk Engine → App: Final scores, categories, decisions, XAI data.**
- **App → Visualisation: Filtered/scored DataFrame.**
- **Visualisation → UI: Plotly figures, formatted table.**

🖼️ **Insert diagram here:**

Title: Figure 2.3 – Information Flow Diagram

Description: A flowchart with labelled arrows: User → Streamlit → Data Loader → Preprocessing → Risk Engine → Visualisation → Streamlit UI.

2.4 Components Design

Component	Responsibility	Technology
data_loader.py	Read CSV, load sample data	Pandas
preprocessing.py	Clean, normalise data	Pandas, NumPy
risk_engine.py	Weighted scoring, rules engine, decision mapping	Pure Python, NumPy
visualization.py	Generate all Plotly charts	Plotly
constants.py	Central config (weights, colours, thresholds)	Python dict
app.py	Streamlit UI orchestration, session management	Streamlit

2.5 Key Design Considerations

- **Explainability: Every risk score is mathematically traceable; business rules are explicit.**
- **Configurability: All feature weights and rule thresholds are adjustable via UI or config file.**
- **Modularity: Core engine is independent of UI, enabling future ML model plug in.**
- **No persistent state: All data lives in memory; no server side database.**
- **Vectorised calculations: Pandas/NumPy ensure fast batch scoring.**

2.6 API Catalogue

The system does not expose external REST APIs. Internal function signatures serve as the API between modules:

Function	Description	Input	Output
preprocess(df)	Clean and normalise	Raw DataFrame	Normalised array
compute_weighted_base(norm, weights)	Base score vector	Array, weights dict	1D array
apply_business_rules(df)	Rule adjustments	DataFrame	(adjustments, triggered)
final_risk(base, adj)	Clipped final scores	arrays	array
categorize(scores)	Risk labels & decisions	array	DataFrame with categories, decisions
generate_explainability(row, weights, rules)	XAI output	customer data	dict of drivers, reducers, rules

These are called in sequence by app.py to refresh all computations.

3. Data Design

3.1 Data Model

The core data entity is the Customer Risk Profile with these logical attributes:

- Identifiers: Customer_ID / Name
- Financial Features: credit_score, payment_history, debt_ratio, monthly_income, existing_loans, fraud_alerts, employment_years, security_flags
- Computed Fields: base_score, final_score, risk_category, decision, triggered_rules

🖼️ Insert sample table here:

Title: Figure 3.1 – Sample Data Model

Description: A table with 5 rows of mock data showing values for each feature and the computed fields.

3.2 Data Access Mechanism

All data resides in a Pandas DataFrame in memory. There is no database; the only persistent data are the CSV uploads and the generated export files. The sample dataset is hard coded in data_loader.py and can be reloaded at any time.

3.3 Data Retention Policies

- **No user data is retained beyond the active browser session.**
- **Uploaded files are processed in memory and not stored on disk.**
- **Generated reports are downloaded by the user and immediately cleared from the server's memory.**

3.4 Data Migration

Not applicable (no persistent storage). If a future version adds a database, migration scripts would convert the static sample dataset and any new customer data.

4. Interfaces

- **User Interface: Streamlit web application, accessible via localhost/cloud.**
- **File Input: CSV file upload (UTF 8).**
- **File Output: CSV download, Excel download (OpenPyXL).**
- **No external system integrations in the current version. Future roadmap includes REST APIs to credit bureaus.**

5. State and Session Management

- **Streamlit's st.session_state dictionary is used to preserve:**
  - **Uploaded data (or sample data)**
  - **Current weight slider values**
  - **Selected customer index**
  - **Global risk category filter**
  - **Flag to control recomputation on weight change**
- **The session is ephemeral and resets when the browser is refreshed or the server restarts.**

6. Caching

- **@st.cache_data is applied to:**
  - **The sample data loading function (to avoid reloading 120 profiles on every interaction).**
  - **Heavy Plotly chart generation functions (portfolio level charts).**
- **Caching is cleared when the user uploads a new file, ensuring fresh computation.**

7. Non Functional Requirements

7.1 Security Aspects

- **Uploaded files are scanned for size limits (e.g., 50 MB) to prevent memory exhaustion.**
- **No personal data is stored or logged; all processing happens in the server's memory.**
- **For deployment in a cloud environment, HTTPS and authentication can be added using Streamlit's built in authentication or a reverse proxy.**
- **No database – reduces risk of data leaks.**

7.2 Performance Aspects

- Average response time for scoring 10,000 records: < 2 seconds.
- UI interactions (weight change, filter) trigger lightweight numpy operations; near instant feedback.
- Charts are generated on demand; first load of complex plots may take ~1s, subsequent loads are cached.
- Memory footprint: ~50 MB for typical dataset.

8. References

- Streamlit Documentation
- Pandas & NumPy User Guides
- Plotly Graphing Library Docs
- OpenPyXL Docs
- Pytest Official Documentation

To create the .docx files:

1. Open Microsoft Word.
2. Paste the LLD content into a new document.
3. Apply your template's formatting (set the font, heading styles, etc.).
4. At the marked points, insert the described screenshots or diagrams (capture them from your running app or draw them with a tool of your choice).
5. Save as LLD_FinRisk_Pro.docx.
6. Repeat the same for the HLD content.

Let me know if you need any adjustments, such as adding more detail to a particular section or refining the algorithm description.